

Informe de Resultados (Breve)

Nombre: Juan Pablo Loja

Fecha: 23/05/2026

Taller 6: Persistencia de Datos en Java – Caso "Taller Padel"

Enlace del Repositorio en GitHub: <https://github.com/juampyz7/POO-E-JuanPabloLoja/tree/main/Taller%206%20-%20Persistencia%20de%20Datos%20en%20Java%20%E2%80%93%20Caso%20Taller%20Padel>

1. ¿Qué sucede si intentas abrir el archivo .dat con un editor de texto?

Cuando intentamos abrir un archivo **.dat** en nuestro editor de texto, ya sea el bloc de notas o cualquier editor de texto básico, lo que vamos a ver es prácticamente son caracteres extraños que son completamente ilegibles para un ser humano. Es una secuencia de caracteres especiales, símbolos de control, cuadros y cualquier otro tipo de características que no corresponden a ninguna palabra que podamos reconocer.

Esto ocurre porque el archivo **.dat** se genera utilizando serialización binaria (**ObjectOutputStream**). Java no guarda el texto literal, sino que convierte la representación interna de cada objeto Java directamente en bytes del estado exacto que tenían los objetos en la memoria. Como nuestro editor de texto intentar hacer la lectura de esos bytes como si fuera un texto normal, el resultado es completamente incomprensible para un ser humano.

Ese comportamiento es completamente esperado y no es que signifique un error. El formato binario está diseñado para ser interpretado exclusivamente por la **JVM**, no por humanos ni por otros programas que no formen parte del ecosistema Java. Su ventaja es la compacidad y la velocidad de lectura y escritura, sacrificando así la legibilidad directa,

2. ¿Cuál es el principal inconveniente de usar .txt frente a .dat cuando el sistema crece en complejidad (muchos atributos o clases relacionadas)?

El principal inconveniente de usar el formato **.txt** es la dificultad del parseo manual y la pérdida de jerarquía de los objetos. A medida que el modelo de datos crece en complejidad, el esfuerzo necesario para mantener el formato de texto crece de forma desproporcionada, mientras que la serialización binaria lo que hace es absorber esos cambios de forma casi automática.

- **Nuevos atributos:** En **.txt** hay que actualizar el formato de línea, el método de guardado y el parseo. En **.dat** es más simple que solo basta con recompilar, la **JVM** detecta eso y serializa los nuevos campos automáticamente.
- **Atributos de tipo objeto anidado:** En **.txt** es necesario tener un sub-formato dentro de cada línea o varias líneas relacionadas, lo cual complica mucho más el parseo. En **.dat** el objeto anidado se serializa recursivamente sin código adicional esto siempre y cuando implemente serializable.

- **Listas internas:** En **.txt** el formato de una línea por partido colapsa, se necesita un esquema completamente nuevo. En **.dat** List, Map y otras colecciones de Java son serializables por defecto.
- **Multiples clases relacionadas:** En **.txt** cada clase requiere su propio esquema de texto, gestión de claves foráneas y lógica de reconstrucción del esquema de objetos. En **.dat** el esquema completo de objeto se serializa y deserializa en una sola operación, conservando las referencias.

Conclusión: Para saber elegir con cual trabajar entre archivos de texto o binarios dependemos directamente de la escalabilidad y los requerimientos de nuestro sistema. Los archivos **.txt** nos ofrecen la ventaja de ser legibles por humanos y útiles para las exportaciones simples, pero resultan ineficientes y muy propensos a errores de formato cuando se maneja varios atributos. Por otro lado, la serialización en archivos binarios **.dat** es la practica mas eficiente dentro la Programación Orientada a Objetos, ya que nos garantiza la persistencia integra de los datos, respeta la encapsulación de las clases y permite guardar o recuperar estructuras de daos complejas de manera directa y segura.